GPU-Accelerated Full-Vocabulary Constraint Decoding

TA: P Pirapuraj

1 Background

Large Language Models (LLMs) are increasingly used for program generation and Automated Program Repair (APR). However, generated programs frequently contain invalid identifiers or API calls that are not available in the current program context. Symbol-table-based constraint decoding can reduce such errors by restricting the tokens that can be generated at each program point.

Consider the following Java code:

```
Calendar calendar = new Calendar();
calendar.
```

At this point, only members that are valid for the object `calendar` should be generated. Suppose the symbol table contains:

```
getTime
getInstance
getFirstDayOfWeek
```

If the LLM generates `getLocale` or `getTimeZone`, these tokens should be rejected because they are not valid members of the object `calendar`.

A straightforward constraint-decoding approach can apply the constraint only to the highest-ranked tokens produced by the language model. Our current implementation uses a **lazy widening strategy**. It first examines the Top-64 tokens and applies the symbolic constraint. If an insufficient number of valid candidates are found, the candidate set is widened to the Top-1024 tokens and the constraint is applied again.

Figure 1 illustrates this process. This strategy reduces the amount of constraint checking in the common case. However, it has two important limitations. First, a valid token may not occur in the Top-64 predictions. For example, if `getTime` is ranked 1025th by the language model, it cannot be recovered even after Top-1024 widening, despite being a valid symbol.

Second, repeatedly performing symbolic constraint checking on large candidate sets can introduce substantial computational overhead. In our initial CPU-based implementation, applying the constraint-decoding process resulted in an additional runtime of more than 23 minutes compared with decoding without constraint checking. This overhead occurs even though the implementation normally begins with only Top-64 candidates and widens to Top-1024 only when necessary.

Checking the complete vocabulary can recover such valid tokens, but a straightforward CPU implementation may be expensive because the vocabulary can contain tens of thousands of tokens. The goal of this project is to investigate whether GPU parallelism can make full-vocabulary constraint decoding efficient.

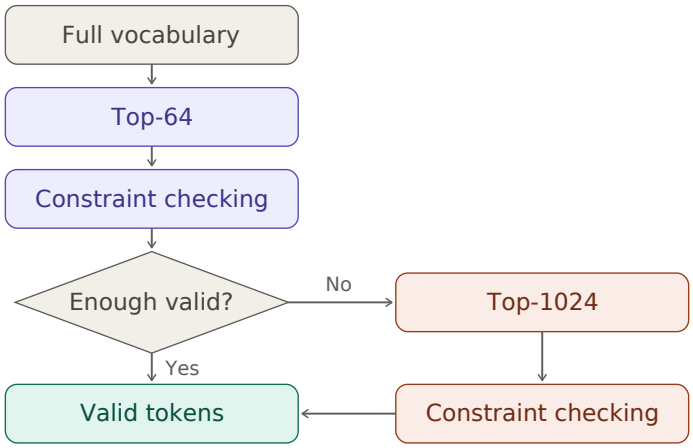


Figure 1: Adaptive top-k constraint filtering.

2 Dataset

Defects4J is a benchmark of reproducible real-world Java bugs for empirical software engineering research. Each entry includes buggy and fixed versions, with tests exposing the defect. Version 1.2 contains 357 bugs, while 2.0 extends to 800+ bugs across more projects, increasing diversity and complexity. The dataset can be accessed via the GitHub page: https://github.com/rjust/defects4j

3 Project Tasks

Students are expected to complete the following tasks as part of this project:

1. **Analyze Candidate Ranking and Lazy Widening:** Analyze the provided dataset to determine how frequently valid program tokens occur within Top-64, within Top-1024, and beyond Top-1024.
   Students should report:
   - Percentage of valid tokens within Top-64.
   - Percentage of valid tokens within Top-1024 but outside Top-64.
   - Percentage of valid tokens outside Top-1024.
2. **Implement Full-Vocabulary CPU Decoding:** Implement a CPU baseline that checks the constraint against every token in the vocabulary.
   Input: Vocabulary, logits, and symbol-table constraint.
   Output: Highest-scoring valid token.
3. **Implement GPU Constraint Checking:** Implement a CUDA kernel that checks the complete vocabulary in parallel. A possible implementation assigns one CUDA thread to each vocabulary token.
   The implementation should generate a validity mask and select the highest-scoring valid token.
4. **Optimize the GPU Implementation:** Investigate at least one optimization such as efficient memory access, shared memory, warp-level operations, parallel reduction, or a compact bit-mask representation of valid tokens.
5. **Evaluation:** Compare the following approaches:
   - Unconstrained decoding.
   - Existing Top-64/Top-1024 lazy-widening CPU decoding.

- Full-vocabulary CPU decoding.
- Full-vocabulary CUDA decoding.
- Optimized CUDA decoding.

The evaluation should measure:

- Constraint-checking latency.
- End-to-end decoding latency.
- GPU speedup over the CPU implementation.
- Number of plausible patches.